IMAGE STEGANOGRAPHY

PROJECT REPORT

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING

By

Batch – B1

G. Himabindu (323103210062)
G. Harshitha (323103210065)
G. Vineetha Devi(323103210081)
J. Srija (323103210086)

Under the esteemed guidance of
Dr. N. Sharmili

Professor

CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]

[Accredited by National Board of Accreditation (NBA) for B. Tech. CSE, ECE & IT – valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]

Kommadi, Madhurawada, Visakhapatnam – 530048

2025 – 2026

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled "**Image Steganography**" is the bonafide work of following III B.Tech. students from the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women, affiliated with AndhraUniversity, Visakhapatnam. This project was done **as part of Cryptography and Network Security lab** during the academic year 2025–2026 (III Year Semester-2).

G. Himabindu (323103210062)
G. Harshitha (323103210065)
G. Vineetha Devi (323103210081)
J. Srija (323103210086)

Signature of the Guide

Dr. N. Sharmili MTech., Ph.D.
Professor,
Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D
Professor,
Dept. of CSE

External Examiner

ABSTRACT

Image steganography is a technique used to securely hide confidential information within digital images without noticeably altering their appearance. This project focuses on implementing an efficient image-in-image steganography system using the concept of Least Significant Bit (LSB) substitution. In this method, a secret image is embedded inside a cover image by modifying the least significant bits of pixel values.

The proposed system enhances traditional steganography by utilizing a **2-bit LSB technique**, which allows more data to be stored compared to the basic 1-bit approach, thereby improving the quality of the extracted image while maintaining acceptable visual similarity of the cover image. The encoding process involves inserting the most significant bits of the secret image into the least significant bits of the cover image. The decoding process reverses this operation to reconstruct the hidden image.

This project is implemented using the Python programming language along with image processing libraries, providing a simple and effective way to demonstrate secure data hiding. The system ensures that the hidden information is not easily detectable, making it useful for applications in secure communication, digital watermarking, and data protection.

Overall, this work highlights the balance between data security, image quality, and embedding capacity in steganographic systems.

1. SYSTEM REQUIREMENTS

1.1. Software Requirements

Software	Version / Specification
Python	3.14.3
Visual Studio Code Editor	1.80
Pillow	10.4.0
Command prompt/terminal	Default system terminal
Operating System	Windows 11

Table1.Software Requirements

Python packages

Python packages we used in our project includes the following table 2.

We used the command pip install “package_name==version” to download the packages.

S.No.	Package Used	Description
1.	NumPy	Used for numerical computations and handling image pixel arrays
2.	Pillow (PIL)	Used for opening, processing, and saving images
3.	Streamlit	Used for developing the web-based user interface (GUI)
4.	Hashlib	Used for generating secure encryption keys using hashing
5.	IO	Used for handling image data streams and enabling file download
6.	Python (Core)	Used for implementing encryption and steganography logic
7.	OS (optional)	Used for file handling and system-level operations (if needed)

Table 2. Python packages

Front End Packages used

S.No.	Package Used	Description
1.	Streamlit	Used for building the web-based user interface and layout
2.	Streamlit Widgets	Used for interactive elements like buttons, file upload, text input, and sidebar
3.	HTML (internal to Streamlit)	Used for structuring the layout of the application interface
4.	CSS(internal styling)	Used for styling and improving the appearance of UI components

Table 3. Front end Packages

1.2 Hardware Requirements

The below Table 4 shows the minimum hardware requirements of our project.

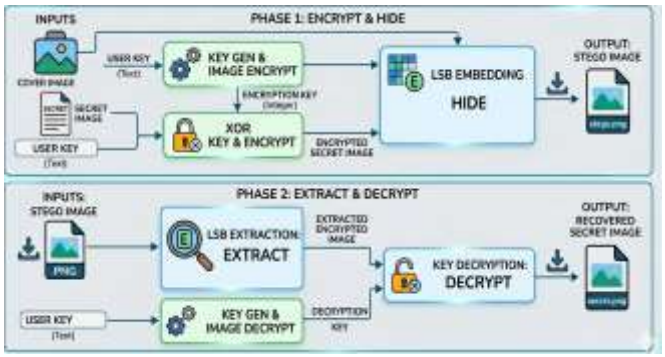
Minimum Hardware Requirements:

S.No.	Hardware Component	Description
1.	Processor	Intel Core i3 or higher (for smooth execution of Python and Streamlit)
2.	RAM	Minimum 4 GB (8 GB recommended for better performance)
3.	Storage	At least 500 MB free space for project files and libraries
4.	System Type	64-bit system recommended
5.	Input Devices	Keyboard and Mouse for user interaction
6.	Display	Monitor with minimum resolution (1366 × 768)

Table 4. Minimum Hardware Requirements

2. METHODOLOGY

2.1 SYSTEM ARCHITECTURE



Components in Detail

1. Client (Streamlit Application)

Role: Presentation Layer — provides user interface

Technology: Python + Streamlit

Functionality:

- Accepts user input (cover image, secret image, key)
- Provides options: **Encrypt & Hide / Extract & Decrypt**
- Sends data to processing functions
- Displays output images (stego image / recovered image)
- Allows downloading of processed images

2. Encryption Module

Role: Security Layer — protects secret image

Technology: Python + NumPy + Hashlib

Functionality:

- Accepts secret image and user key
- Converts key into numeric value using hashing (SHA-256)
- Applies XOR encryption on image pixels
- Produces encrypted image data

3. Steganography Module (LSB Encoder/Decoder)

Role: Data Hiding Layer — embeds and extracts image

Technology: NumPy + Pillow

Functionality:

Encoding:

- Takes cover image and encrypted secret image
- Embeds secret image using **2-bit LSB technique**
- Generates stego image

Decoding:

- Extracts hidden bits from stego image
- Reconstructs encrypted image
- Sends to decryption module

4. Decryption Module

Role: Recovery Layer — retrieves original image

Technology: Python + NumPy

Functionality:

- Takes extracted encrypted image
- Uses same key for XOR decryption
- Recovers original secret image

5. Image Processing Module

Role: Utility Layer — handles image operations

Technology: Pillow (PIL)

Functionality:

- Opens and converts images (RGB format)
- Resizes images if required
- Saves images in proper format (PNG)
- Ensures valid image processing

Communication Flow (Step by Step)

1. User - Client

- User uploads:
  1. Cover image
  2. Secret image
  3. Secret key
- Selects operation (Encrypt & Hide)

2. Client - Encryption Module

- Secret image is passed for encryption
- Key is converted using hashing
- Encrypted image is generated

3. Encryption - Steganography

- Encrypted image is embedded into cover image
- Using **2-bit LSB technique**
- Stego image is created

4. Steganography - Client

- Stego image is displayed
- User downloads stego image

5. Client - Steganography (Decoding)

- User uploads stego image
- Hidden image is extracted

6. Steganography - Decryption

- Extracted image is decrypted using same key
- Original secret image is recovered

7. Client (UI)

- Displays recovered secret image
- Allows download

Architecture Type: This system follows a **three-layer architecture:**

Layer	Component
Presentation Layer	Streamlit UI
Processing Layer	Encryption + Steganography
Data Layer	Image Data (Cover & Secret)

Performance Characteristics

- System runs locally (no network dependency)
- Fast processing using NumPy arrays
- Requires correct key for decryption
- Works best with PNG images (lossless)
- Secret image size must be ≤ cover image size

2.2 DESCRPTION

Image Steganography using 2-bit LSB technique is a method of hiding a secret image inside a cover image without visibly changing the original image.

In this project, the Least Significant Bits (LSB) of the cover image pixels are modified to store the most significant bits of the secret image. Since only the last bits are changed, the difference is not noticeable to the human eye.

The system performs two main operations:

- **Encoding:** Hides the secret image inside the cover image
- **Decoding:** Extracts the hidden image from the stego image

The implementation is done using Python and the Pillow library for image processing.

ALGORITHM

Encoding Algorithm (2-bit LSB)

1. Start
2. Load the cover image
3. Load the secret image
4. Check if secret image size ≤ cover image size
5. Convert both images into pixel values (RGB)
6. For each pixel:
    - Take RGB values of cover image
    - Take RGB values of secret image
    - Extract **top 2 bits** of secret image
    - Replace **last 2 bits** of cover image with secret bits
7. Store modified pixels in a new image
8. Save the output as **stego image**
9. Stop

Decoding Algorithm (2-bit LSB)

1. Start
2. Load the stego image
3. Extract pixel values (RGB)
4. For each pixel:
    - Extract **last 2 bits** from RGB values
    - Shift bits to reconstruct original pixel
5. Store reconstructed pixels in new image
6. Save as **secret image output**
7. Stop

3. IMPLEMENTATION

3.1Source code

```
import streamlit as st
from PIL import Image
import numpy as np
import hashlib
import io

# ---------------- KEY GENERATION ----------------
def generate_key(user_key):
    user_key = str(user_key)
    hash_object = hashlib.sha256(user_key.encode())
    key = int(hash_object.hexdigest(), 16) % 256
    return key

# ---------------- ENCRYPTION ----------------
def encrypt_image(img_array, key):
    key = generate_key(key)
    return (img_array ^ key).astype('uint8')

def decrypt_image(img_array, key):
    key = generate_key(key)
    return (img_array ^ key).astype('uint8')

# ---------------- SAFE IMAGE LOADER ----------------
def load_image(uploaded_file):
    try:
        uploaded_file.seek(0)
        return Image.open(uploaded_file).convert("RGB")
    except:
        st.error("❌ Unable to read image. Upload valid PNG/JPG.")
        return None

# ---------------- STEGANOGRAPHY ----------------
def encode_image(cover, secret):
    cover = np.array(cover, dtype=np.uint8)
    secret = np.array(secret, dtype=np.uint8)
```

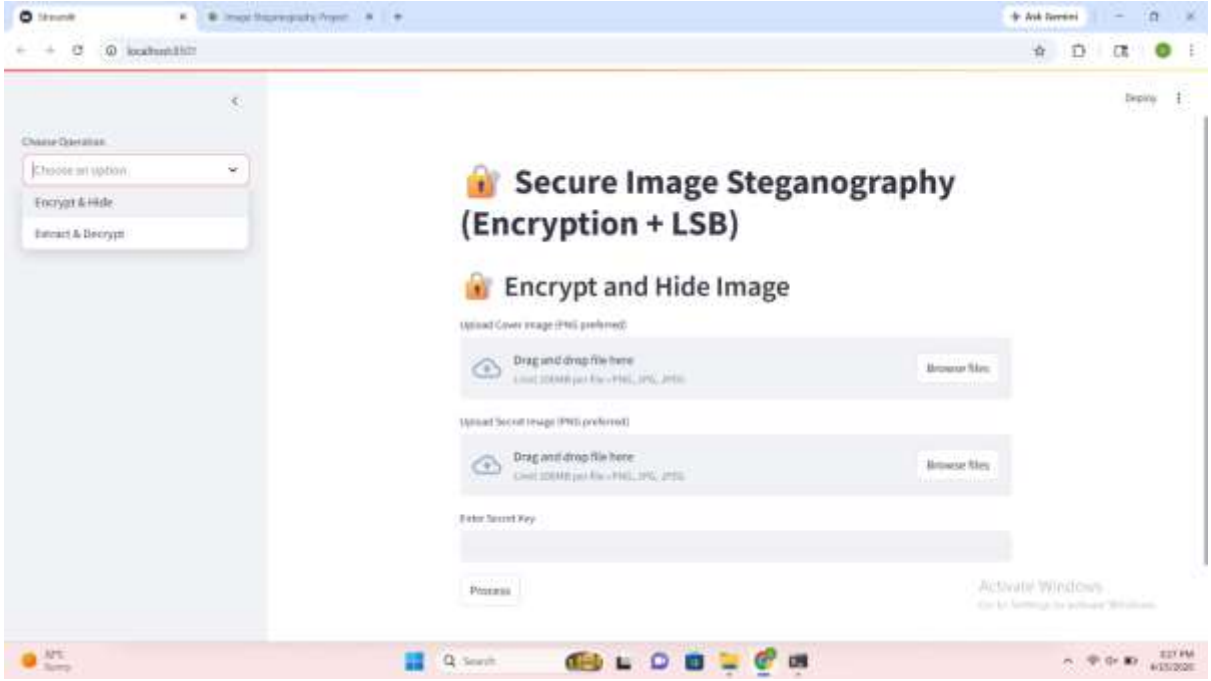
```
    h, w, _ = cover.shape
    sh, sw, _ = secret.shape

    if sh > h or sw > w:
        st.error("❌ Secret image must be smaller than cover image!")
        return None

    encoded = cover.copy()

    for i in range(sh):
        for j in range(sw):
            for k in range(3):
                encoded[i][j][k] = (cover[i][j][k] & 0xFC) | (secret[i][j][k] >> 6)

    return Image.fromarray(encoded.astype('uint8'))

def decode_image(stego):
    stego = np.array(stego, dtype=np.uint8)
    extracted = np.zeros_like(stego)

    for i in range(stego.shape[0]):
        for j in range(stego.shape[1]):
            for k in range(3):
                extracted[i][j][k] = (stego[i][j][k] & 3) << 6

    return Image.fromarray(extracted.astype('uint8'))

# ---------------- UI ----------------
st.title("🔐 Secure Image Steganography (Encryption + LSB)")

option = st.sidebar.selectbox("Choose Operation", ["Encrypt & Hide", "Extract & Decrypt"])

# ---------------- ENCRYPT ----------------
if option == "Encrypt & Hide":
    st.header("🔐 Encrypt and Hide Image")

    cover_file = st.file_uploader("Upload Cover Image (PNG preferred)", type=["png", "jpg", "jpeg"])
```

```
secret_file = st.file_uploader("Upload Secret Image (PNG preferred)", type=["png", "jpg", "jpeg"])
key = st.text_input("Enter Secret Key")

if st.button("Process"):
    if cover_file and secret_file and key:

        cover = load_image(cover_file)
        secret = load_image(secret_file)

        if cover and secret:
            # Resize secret to cover size (important)
            secret = secret.resize(cover.size)

            # Encrypt
            encrypted = encrypt_image(np.array(secret), key)
            encrypted_img = Image.fromarray(encrypted)

            # Hide
            stego = encode_image(cover, encrypted_img)

            if stego:
                st.image(stego, caption="Stego Image")

                # Save properly
                buf = io.BytesIO()
                stego.save(buf, format="PNG")

                st.download_button(
                    "📥 Download Stego Image",
                    buf.getvalue(),
                    "stego.png",
                    "image/png"
                )

                st.success("✅ Image encrypted and hidden!")

    else:
        st.warning("⚠ Upload images and enter key")
```

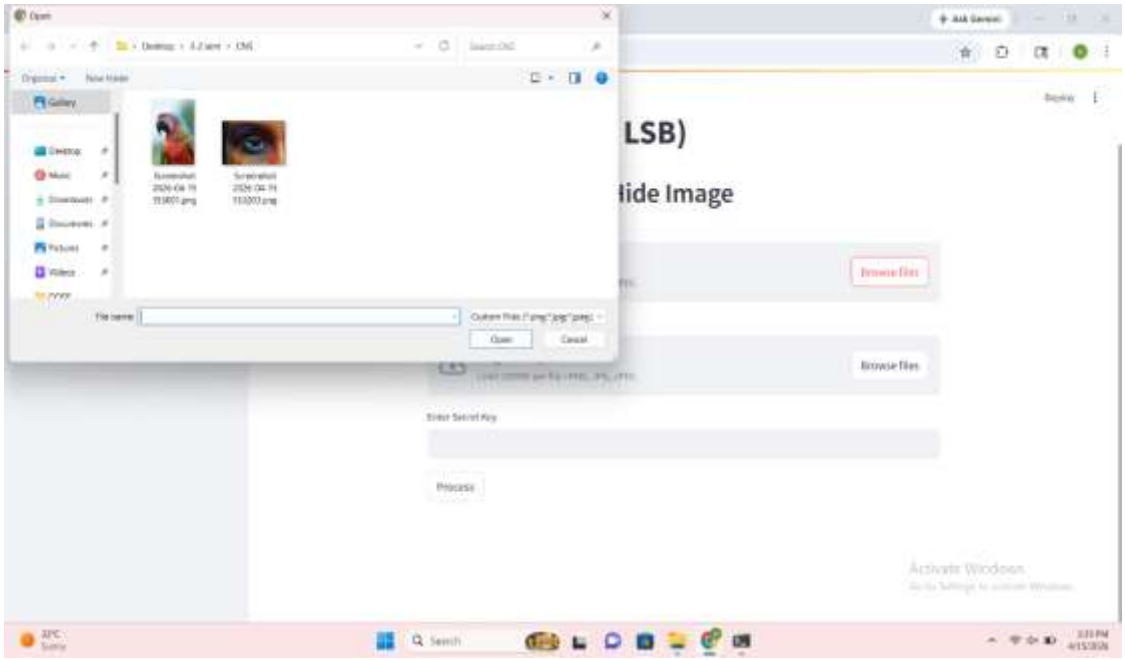
```
# ---------------- DECRYPT ----------------
elif option == "Extract & Decrypt":
    st.header("🔓 Extract and Decrypt Image")

    stego_file = st.file_uploader("Upload Stego Image", type=["png", "jpg", "jpeg"])
    key = st.text_input("Enter Secret Key")

    if st.button("Process"):
        if stego_file and key:

            stego = load_image(stego_file)

            if stego:
                # Extract
                extracted = decode_image(stego)

                # Decrypt
                decrypted = decrypt_image(np.array(extracted), key)
                final_img = Image.fromarray(decrypted)

                st.image(final_img, caption="Recovered Secret Image")

                # Save properly
                buf = io.BytesIO()
                final_img.save(buf, format="PNG")

                st.download_button(
                    "📥 Download Secret Image",
                    buf.getvalue(),
                    "secret.png",
                    "image/png"
                )

                st.success("✅ Image extracted and decrypted!")

        else:
            st.warning("⚠ Upload image and enter key")
```

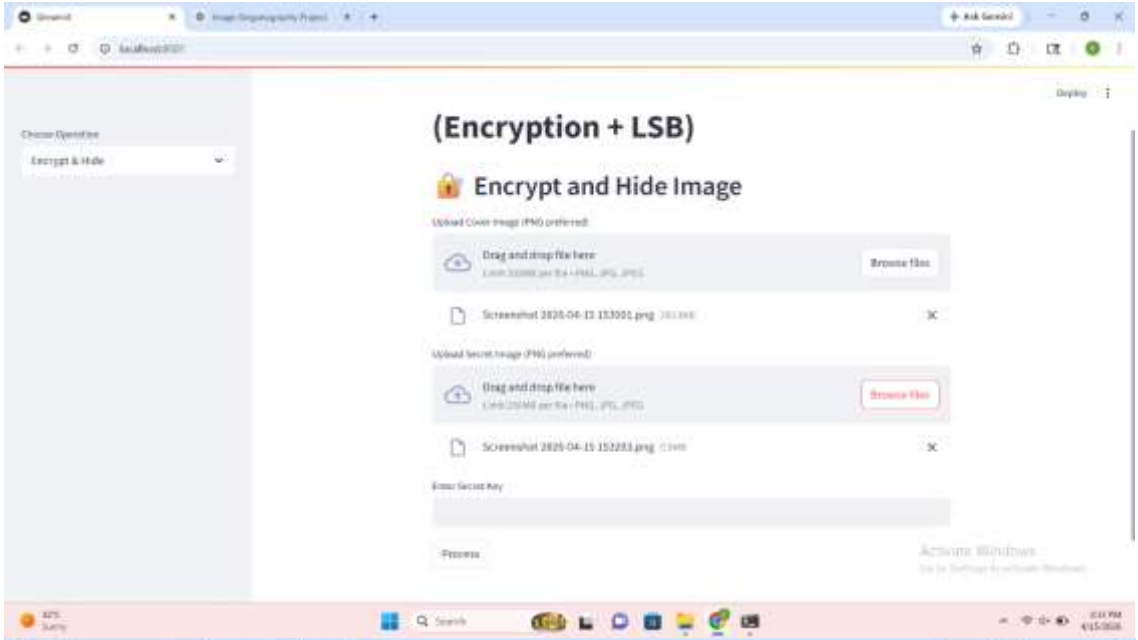
4. RESULTS

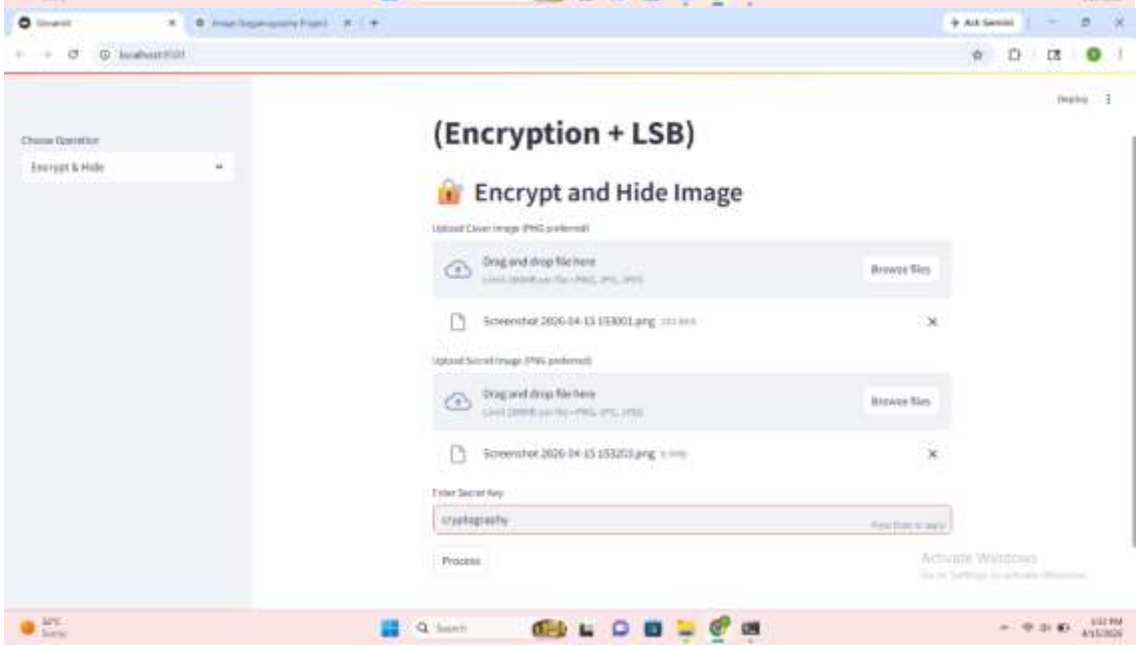
4.1 Output Screens

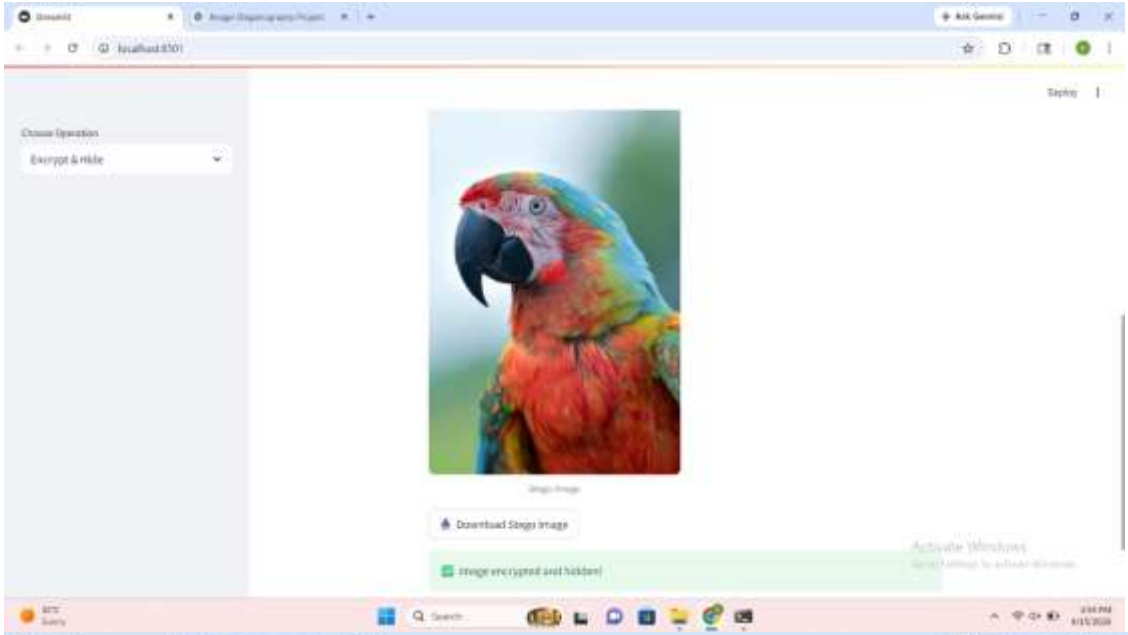


Secure Image Steganography
(Encryption + LSB)
Encrypt and Hide Image
Encrypt & Hide
Drag and drop file here
Browse files
Drag and drop file here
Browse files
Process

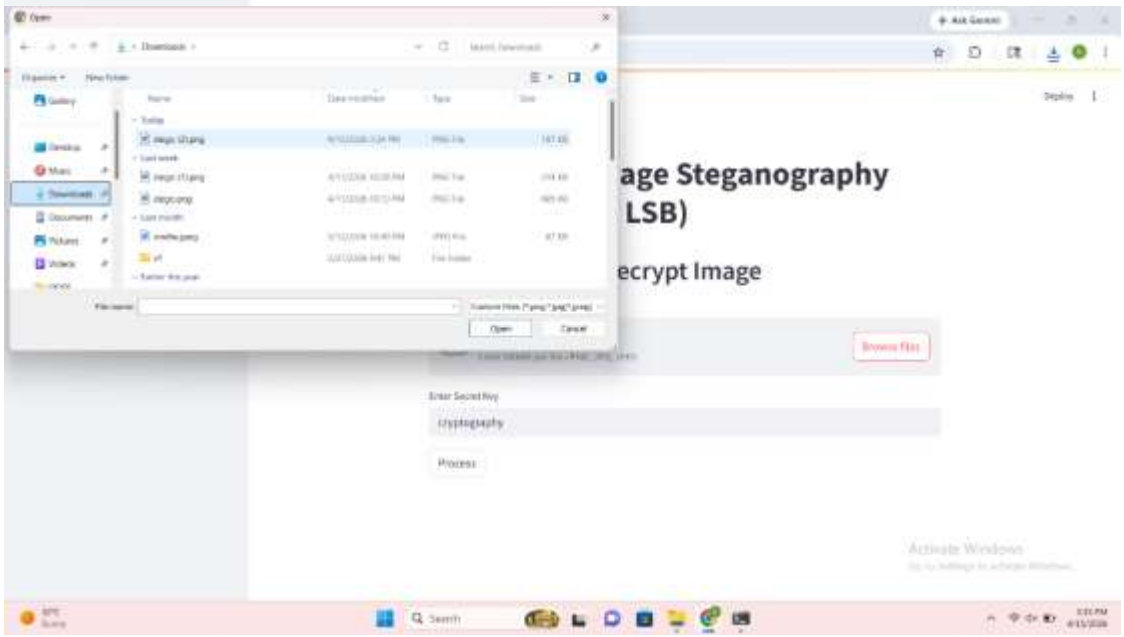

LSB)
Hide Image
Browse files
Browse files
Process

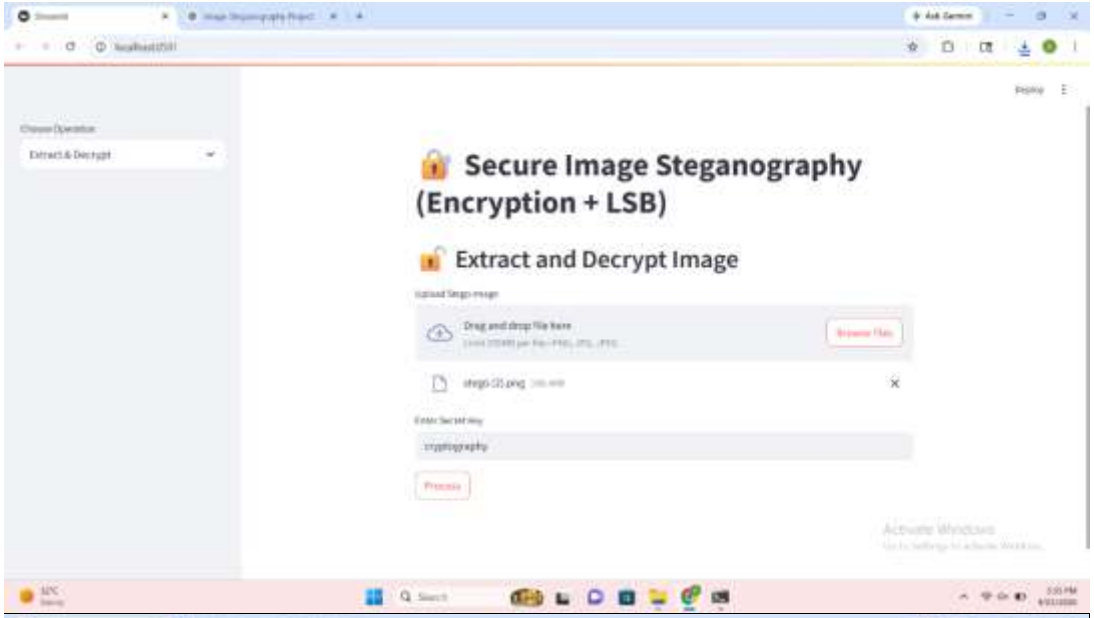

(Encryption + LSB)
Encrypt and Hide Image
Drag and drop file here
Browse files
Process

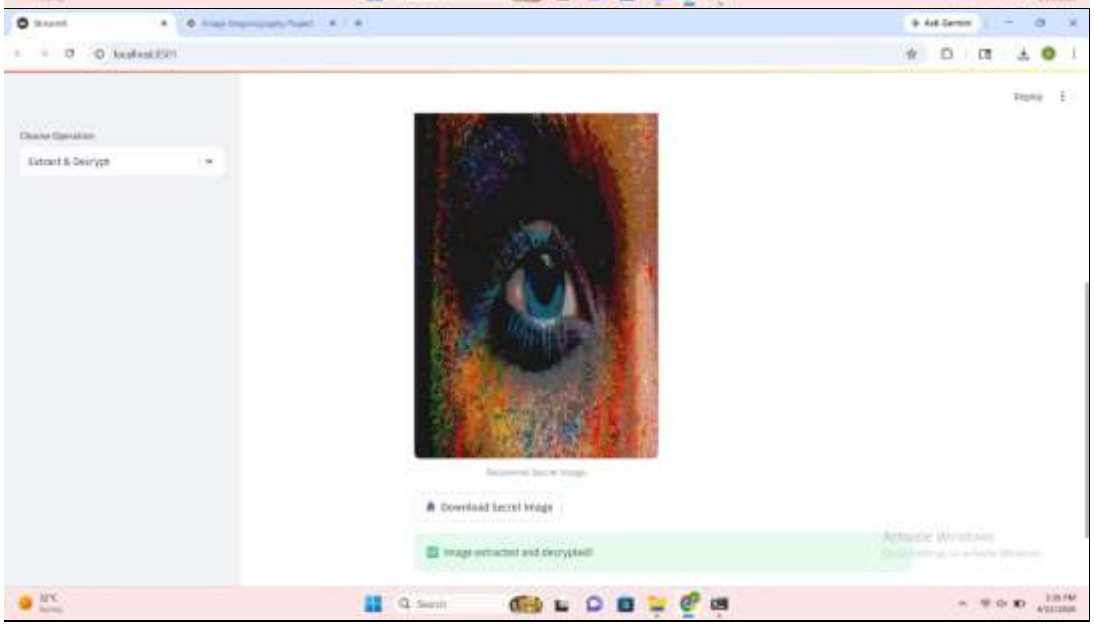

(Encryption + LSB)
Encrypt and Hide Image
Drag and drop file here
Browse files
Process


Encrypt & Hide
Download Stego Image
Image encrypted and hidden

Extract & Decrypt
Encrypt & Hide
Extract & Decrypt
Secure Image Steganography
(Encryption + LSB)
Extract and Decrypt Image
Drag and drop file here
Browse files
cryptography
Process


cryptography
Process


Secure Image Steganography
(Encryption + LSB)
Extract and Decrypt Image
Extract & Decrypt
Drag and drop file here
cryptography
Process


Extract & Decrypt
Download Secret Image
Image extracted and decrypted!

5. CONCLUSION

This project successfully demonstrates the implementation of **secure image steganography** by combining **encryption and LSB (Least Significant Bit) techniques**. The system allows a secret image to be encrypted using a user-defined key and then hidden inside a cover image without visibly altering its appearance.

By integrating **cryptography with steganography**, the project ensures enhanced security, as the hidden data remains both invisible and protected. Even if the hidden data is extracted, it cannot be understood without the correct key.

The use of **2-bit LSB technique** improves the quality of the extracted image compared to basic methods, maintaining a balance between **image clarity and data security**. Additionally, the implementation of a **Streamlit-based user interface** makes the system user-friendly and interactive.

Overall, this project highlights the importance of secure data transmission and demonstrates how combining multiple security techniques can provide a more robust solution for protecting sensitive information.

6. FUTURE SCOPE

The proposed system of secure image steganography can be further enhanced in several ways to improve its performance and security. In the future, stronger encryption algorithms such as AES or RSA can be implemented instead of basic XOR encryption to provide higher security. The use of advanced or adaptive LSB techniques can improve the quality of the extracted image while maintaining the invisibility of hidden data. Additionally, the system can be extended to support multiple types of data such as text, audio, and video instead of only images. Incorporating user authentication or password-based access control can further enhance security. The project can also be developed into a mobile application or integrated with cloud platforms for real-time and secure data sharing. Furthermore, the application of artificial intelligence and machine learning techniques can help in improving data hiding efficiency and detecting steganographic attacks. Overall, these enhancements can make the system more robust, secure, and suitable for real-world applications.

7. REFERENCES

Books Referred:

1. N. Provos and P. Honeyman, "Hide and Seek: An Introduction to Steganography," IEEE Security & Privacy, 2003.
2. William Stallings, "Cryptography and Network Security: Principles and Practice," Pearson Education.

Journal Articles:

1. J. Fridrich, "Steganography in Digital Media: Principles, Algorithms, and Applications," Cambridge University Press, 2009.
2. M. Hussain et al., "A Survey of Image Steganography Techniques," International Journal of Advanced Science and Technology, vol. 54, pp. 113–124, 2013.

Online Resources:

1. GeeksforGeeks. "Image Steganography using Python." [Online]. Available: https://www.geeksforgeeks.org/image-steganography-using-python/
2. Wikipedia. "Steganography." [Online]. Available: https://en.wikipedia.org/wiki/Steganography
3. Streamlit Documentation. "Streamlit Library." [Online]. Available: https://docs.streamlit.io/
4. www.google.com
5. www.wikipedia.com